Núcleo

Informe de avance y guía de integración

Gestión y monitoreo del ambiente

Fecha: 24 de octubre de 2026

Alcance: backend, bases de datos, broker MQTT, canal en vivo, firmware, web, app móvil y repositorio.

Para qué sirve: es una mini guía para que cualquier integrante entienda en qué punto está el proyecto, qué hace cada carpeta y cada archivo, cómo levantar el entorno en minutos, cómo subir cambios al repositorio y qué falta definir en el SRS para terminar el backend, el IoT y Mosquitto.

Contenido

Contenido 2

1. Resumen del avance 4

1.1 Estado por componente . . . 4

1.2 Cifras verificadas . . . 4

2. Arquitectura y flujos 6

2.1 Comunicación entre componentes . . . 6

2.2 Servicios y puertos del entorno local . . . 6

2.3 Qué ocurre cuando llega una lectura . . . 6

3. Guía rápida para integrarse 8

3.1 Requisitos . . . 8

3.2 Variables de entorno (los .env no se versionan) . . . 8

3.3 Primer arranque, paso a paso . . . 8

3.4 Problemas frecuentes . . . 9

4. Mapa del proyecto: carpetas y archivos 10

4.1 Raíz del proyecto (Nucleo/) . . . 10

4.2 BackEnd/ (raíz del backend) . . . 10

4.3 BackEnd/config/ . . . 10

4.4 BackEnd/common/ . . . 11

4.5 apps/core/ . . . 11

4.6 apps/catalogos/ . . . 11

4.7 apps/organizaciones/ . . . 11

4.8 apps/usuarios/ . . . 12

4.9 apps/dispositivos/ . . . 12

4.10 apps/umbrales/ . . . 12

4.11 apps/mediciones/ . . . 12

4.12 apps/alertas/ . . . 13

4.13 apps/reportes/ . . . 13

4.14 apps/tiempo_real/ . . . 13

4.15 FrontEnd/ (web React + Vite) . . . 13

4.16 Mobile/ (app Expo + React Native) . . . 14

4.17 IoT/ (ESP32 con MicroPython) . . . 14

4.18 Mosquitto/ (broker MQTT) . . . 15

4.19 Database/ (scripts SQL y roles) . . . 15

4.20 Context/ (documentación técnica) . . . 16

4.21 Docs/ (documentos del proyecto) . . . 16

5. Cómo funciona por dentro 17

5.1 Modelo de datos . . . 17

5.2 Roles y alcance de los datos . . . 17

5.3 API REST (prefijo /api/v1/) . . . 18

5.4 Token y sesión . . . 18
5.5 Canal en vivo (WebSocket) . . . 18
5.6 MQTT y Mosquitto . . . 19
5.7 Firmware del ESP32 . . . 19
5.8 Reportes . . . 20
5.9 Correo y notificaciones push . . . 20
5.10 Web y app móvil: cómo repartir el trabajo . . . 20
5.11 Decisiones y diferencias a tener presentes . . . 21
6. Base de datos y pruebas locales 22
6.1 Los tres espacios . . . 22
6.2 Comandos habituales . . . 22
6.3 Recrear una base de desarrollo (destructivo) . . . 22
6.4 Usuarios de base de datos sin superusuario . . . 22
6.5 Cosas a tener en cuenta en las pruebas locales . . . 22
7. Estado de verificación 24
7.1 Verificado . . . 24
7.2 No verificado todavía . . . 24
7.3 Cómo repetir la prueba extremo a extremo . . . 24
8. Repositorio y flujo de trabajo en Git 25
8.1 Las ramas y para qué sirve cada una . . . 25
8.2 Qué se sube y qué no . . . 25
8.3 Cómo subir cambios, con ejemplos . . . 25
8.4 Mensajes de commit . . . 26
8.5 Antes de abrir un pull request . . . 27
8.6 Acceso a GitHub y seguridad . . . 27
8.7 Estado actual del repositorio . . . 27
9. Pendientes para integrar al SRS 28
9.1 Backend . . . 28
9.2 IoT (firmware y hardware) . . . 29
9.3 Mosquitto (broker MQTT) . . . 29
9.4 Base de datos y pruebas locales . . . 29
9.5 Transversales . . . 30
9.6 Orden sugerido para cerrar la brecha . . . 30

1. Resumen del avance

Núcleo mide temperatura, humedad y calidad del aire con nodos ESP32, guarda las lecturas, avisa cuando algo sale de los límites y ofrece una API compartida para una web y una app móvil. Este informe reúne lo construido hasta el 24 de octubre de 2026, explica cada carpeta y archivo del proyecto, indica cómo integrarse en pocos minutos y termina con la lista de temas que faltan por definir en el SRS para cerrar el backend, el IoT y el broker.

En una frase: **el backend, el modelo de datos, el broker y el canal en vivo están completos y verificados; el firmware está escrito y probado en lógica pero no en la placa; la web y la app móvil son un esqueleto de módulos enlazados listo para repartir el trabajo.**

1.1 Estado por componente

Componente	Estado	Qué hay hoy	Qué falta
Backend (Django + DRF)	Completo y verificado	API de organizaciones, áreas, usuarios, dispositivos, sensores, umbrales, lecturas, series, alertas, notificaciones, reportes y programaciones; permisos por rol; aislamiento por organización; token con caducidad; ingesta MQTT; alertas por nivel; reportes con PDF y gráficas; canal en vivo; push con Expo.	Proveedor de IA y de correo reales; agente de IA; probar el push con un teléfono real; despliegue.
Bases de datos	Completo y verificado	PostgreSQL 17 con 16 tablas (`nucleo_db`) y TimescaleDB con la hypertable `lectura` (`nucleo_iot`), creadas por migraciones; scripts de práctica; `crear-roles.sh`.	Aplicar los usuarios sin superusuario en el despliegue; retención y respaldos.
Broker MQTT (Mosquitto)	Completo y verificado	TLS en 8883, un usuario por dispositivo, ACL por usuario (`%u`), scripts `preparar.sh` y `dispositivo.sh`, acceso desde la red con `MQTT_BIND` y `MQTT_SERVER_NAMES`.	Certificados y secretos de producción; alta de dispositivos desde la plataforma.
Canal en vivo	Completo en el backend	WebSocket `ws/panel/?token=` con Redis; eventos de alertas y estado de dispositivos filtrados por rol.	Que la web y la app lo consuman.
Firmware ESP32	Escrito; sin probar en hardware	DHT22, CCS811 con calentamiento y baseline, OLED, LED RGB, buzzer, Wi-Fi, NTP, MQTT con TLS, cola de lecturas sin conexión y 16 pruebas de lógica.	Probar en la placa (pines, TLS, batería) y calibrar el CCS811.
Web (React)	Esqueleto	Menú y rutas para 8 módulos con «Hola mundo…», estado de la API, cliente HTTP.	Implementar cada módulo, sesión y canal en vivo.
App móvil (Expo)	Esqueleto	Navegación y pantallas para 5 módulos con «Hola mundo…», estado de la API, cliente HTTP.	Implementar cada módulo, sesión, push y canal en vivo.
Repositorio (GitHub)	Publicado	`BrandonGlz/Nucleo` con `main`, `dev` y una rama por área (`backend`, `frontend`, `mobile`, `iot`); guía de ramas y commits en el README.	Protección de ramas, integración continua y tags por etapa.
Documentación	Al día	READMEs por carpeta, `Context/` completo, SRS y este informe.	Reportes formales de avance y seguridad en `Docs/`.

1.2 Cifras verificadas

Comprobación	Resultado	Cómo repetirla
Pruebas del backend	134 pruebas, todas correctas (usan bases temporales `test_nucleo_db` y `test_nucleo_iot`)	`cd BackEnd && .venv/bin/python manage.py test --noinput`
Estilo del backend	Ruff sin avisos; `manage.py check` sin problemas; sin migraciones pendientes	`.venv/bin/ruff check .` y `manage.py makemigrations --check --dry-run`
Pruebas del firmware	16 pruebas correctas en la computadora	`IoT/.venv/bin/python -m unittest discover -s IoT/tests -t IoT`
Web	Oxlint sin avisos; `vite build` correcto	`cd FrontEnd && npm run lint && npm run build`
App móvil	Oxlint sin avisos; `expo-doctor` 20/20; dependencias alineadas con SDK 58; bundle de Android de 872 módulos	`cd Mobile && npm run lint && npm run export:android`
Extremo a extremo	Un dispositivo publicó por TLS con su usuario, el consumidor guardó 3 lecturas, creó la alerta de Peligro y el evento llegó por Redis a un WebSocket abierto sin tráfico durante 20 s	Procedimiento en la sección 7

Comprobación	Resultado	Cómo repetirla
Aislamiento del broker	Un dispositivo no puede publicar en el tópico de otro aunque cambie su client ID	Procedimiento en la sección 7

2. Arquitectura y flujos

Hay un solo backend. La web y la app móvil son dos clientes de la misma API REST y del mismo canal WebSocket. El navegador y el teléfono nunca hablan con las bases de datos ni con el broker MQTT.

2.1 Comunicación entre componentes

Origen	Destino	Protocolo	Estado
Web	Backend	HTTP/JSON con `Authorization: Token`	API lista; la web solo consulta la salud
App móvil	Backend	HTTP/JSON con token	API lista; la app solo consulta la salud
Web y app	Backend	WebSocket `/ws/panel/?token=`	Activo en el backend; sin cliente todavía
ESP32	Mosquitto	MQTT sobre TLS, puerto 8883	Firmware escrito; sin probar en hardware
Mosquitto	Backend (`escuchar_mqtt`)	MQTT, suscripción a `nucleo/devices/+/measurements`	Activo
Backend	Mosquitto	MQTT retenido en `nucleo/devices/<codigo>/config`	Activo con `MQTT_PUBLICAR_CONFIG`
Backend	PostgreSQL `nucleo_db`	SQL (psycopg 3)	Activo: 16 tablas
Backend	TimescaleDB `nucleo_iot`	SQL (psycopg 3)	Activo: hypertable `lectura`
Backend	Redis	Capa de canales (`channels-redis`)	Activo
Backend	Expo Push	HTTPS a `exp.host`	Implementado; apagado por defecto
Backend	Correo (SMTP)	SMTP	Activo; en desarrollo se imprime en consola
Backend	Servicio de IA	HTTPS	Proveedor por definir; sin él los reportes responden 503

2.2 Servicios y puertos del entorno local

Servicio	Dirección	Contenedor / comando	Notas
API y WebSocket de Django	127.0.0.1:8000	`manage.py runserver 0.0.0.0:8000`	Daphne atiende HTTP y WebSocket
Web (Vite)	localhost:5173	`npm run dev`	Puerto fijo; el backend permite ese origen por CORS
App móvil (Expo)	QR de Expo	`npm start`	En el teléfono, la API se alcanza por la IP local
PostgreSQL 17	127.0.0.1:55432	`nucleo_postgres`	Base `nucleo_db`, usuario `nucleo_app`
TimescaleDB 2.31	127.0.0.1:55433	`nucleo_timescaledb`	Base `nucleo_iot`, usuario `nucleo_iot`
Redis 7	127.0.0.1:56379	`nucleo_redis`	Con contraseña; sin persistencia
Mosquitto 2	127.0.0.1:8883 (`MQTT_BIND`)	`nucleo_mosquitto`	Solo TLS; para el ESP32 se abre a la red
PostgreSQL 18 nativo	127.0.0.1:5432	`postgresql-18.service`	Ajeno a Núcleo; no se toca

2.3 Qué ocurre cuando llega una lectura

- **El ESP32** lee DHT22 y CCS811, clasifica con la última configuración recibida, enciende LED y buzzer, y publica un JSON en `nucleo/devices/<codigo_mqtt>/measurements`. Si no hay conexión, guarda el mensaje (con hora) en una cola de 20 y lo envía al volver.
- **Mosquitto** autentica al dispositivo con su usuario y solo le deja publicar en su propio tópico.
- `escuchar_mqtt` valida el JSON, el dispositivo, el sensor, la variable y el rango físico. Una lectura inválida se descarta y el resto del mensaje se guarda.
- **TimescaleDB** recibe cada lectura con su hora de medición y de recepción; el aire del CCS811 se marca `estabilizada = false` durante los 20 minutos de calentamiento.

- **El dispositivo** actualiza última conexión, batería y estado (`LINEA`, `CALENT`, `BATBAJA`), y se emite `dispositivo.estado` si cambió.
- **Cada lectura estabilizada** se clasifica con el umbral activo: Normal dentro de los límites de advertencia, Advertencia entre estos y los de peligro, Peligro más allá.
- **La alerta** se crea en `nucleo_db` con una copia de la lectura (no hay llave foránea entre bases) y una notificación por responsable y canal, en una sola transacción. Mientras haya una pendiente del mismo umbral y nivel no se crea otra; si una advertencia escala a peligro se crea una nueva.
- **Al confirmarse la transacción** se envía el correo al supervisor del área (o a los administradores), se envía el push si está activo y se publica `alerta.nueva` en Redis.
- **El consumidor WebSocket** reenvía el evento a las conexiones de esa organización que su rol puede ver.

Otros flujos: la configuración (intervalo y umbrales) viaja en sentido contrario como mensaje retenido para que el nodo la use incluso sin conexión; `marcar_sin_conexion` cambia a «Sin conexión» a los dispositivos silenciosos más de tres intervalos; y los reportes calculan las cifras en SQL, piden a la IA solo la redacción y generan el PDF, que las programaciones envían por correo.

3. Guía rápida para integrarse

Con esta sección una persona nueva puede tener todo corriendo en el equipo. Todos los comandos se ejecutan desde la carpeta principal del proyecto.

3.1 Requisitos

Herramienta	Versión usada	Para qué
Python	3.12	Backend e IoT (el sistema también tiene 3.14; no se usa)
Node.js / npm	24 / 11	Web y app móvil
Docker Engine / Compose	29 / 5	PostgreSQL, TimescaleDB, Redis y Mosquitto; el usuario debe estar en el grupo `docker`
Git	2.5x	Control de versiones
VS Code + Pymakr	Pymakr 2.22	Editar y subir el firmware (opcional: `mpremote`)
ESP32 + sensores	DevKit, DHT22, CCS811, OLED	Solo para probar el firmware; el resto funciona sin hardware

3.2 Variables de entorno (los `.env` no se versionan)

```
cp .env.example .env
cp BackEnd/.env.example BackEnd/.env
cp FrontEnd/.env.example FrontEnd/.env
cp Mobile/.env.example Mobile/.env
chmod 600 .env BackEnd/.env FrontEnd/.env Mobile/.env
openssl rand -hex 24      # una contraseña distinta por cada valor
```

Archivo	Variable	Qué es
.env (raíz)	`POSTGRES_PASSWORD`, `IOT_PASSWORD`, `REDIS_PASSWORD`	Contraseñas de los contenedores. Deben repetirse dentro de las URL de `BackEnd/.env`.
.env (raíz)	`POSTGRES_PORT`, `IOT_PORT`, `REDIS_PORT`, `MQTT_PORT`	Puertos locales (55432, 55433, 56379, 8883).
.env (raíz)	`MQTT_BACKEND_PASSWORD`, `MQTT_DEVICE_PASSWORD`	Contraseñas de `nucleo_backend` y del usuario `NUC-001` en el broker.
.env (raíz)	`MQTT_BIND`, `MQTT_SERVER_NAMES`	Para exponer el broker a la red: `0.0.0.0` y la IP o nombre que usará el ESP32.
.env (raíz)	`BACKEND_DB_PASSWORD`, `BACKEND_IOT_DB_PASSWORD`	Solo para `Database/crear-roles.sh` (usuarios sin superusuario).
BackEnd/.env	`DJANGO_SECRET_KEY`, `DJANGO_ALLOWED_HOSTS`, `DJANGO_DEBUG`	Clave de Django, hosts permitidos y modo desarrollo.
BackEnd/.env	`DATABASE_URL`, `IOT_DATABASE_URL`, `REDIS_URL`	Conexiones a las dos bases y a Redis (vacío en `REDIS_URL` = capa en memoria).
BackEnd/.env	`MQTT_HOST`, `MQTT_USERNAME`, `MQTT_PASSWORD`, `MQTT_USE_TLS`, `MQTT_CA_CERT`	Cliente del backend hacia el broker (`nucleo_backend` y la CA en `Mosquitto/certs/ca.crt`).
BackEnd/.env	`CORS_ALLOWED_ORIGINS`, `CSRF_TRUSTED_ORIGINS`	Origen de la web (5173).
BackEnd/.env	`NUCLEO_TOKEN_DIAS_SIN_USO`, `NUCLEO_PUSH_HABILITADO`, `NUCLEO_SIN_CONEXION_FACTOR`, `NUCLEO_BATERIA_BAJA_PCT`	Caducidad del token (7), push (False), intervalos sin datos para «Sin conexión» (3) y batería baja (20 %).
BackEnd/.env	`EMAIL_*`, `IA_PROVEEDOR`, `IA_API_KEY`, `IA_MODELO`	Correo (consola por defecto) e IA (sin proveedor).
FrontEnd/.env	`VITE_API_BASE_URL`	`http://localhost:8000/api/v1`.
Mobile/.env	`EXPO_PUBLIC_API_BASE_URL`	`http://10.0.2.2:8000/api/v1` en emulador Android; la IP local en un teléfono.

3.3 Primer arranque, paso a paso

#	Comando	Qué hace	Resultado esperado
1	`docker compose up -d --wait`	Enciende PostgreSQL, TimescaleDB y Redis (y Mosquitto si ya tiene certificados)	Los contenedores quedan `healthy`

#	Comando	Qué hace	Resultado esperado
2	`python3.12 -m venv BackEnd/.venv` y `BackEnd/.venv/bin/pip install -r BackEnd/requirements-dev.txt`	Entorno Python del backend	Instalación sin errores
3	`cd BackEnd && .venv/bin/python manage.py migrate`	Crea las 16 tablas y carga los catálogos en `nucleo_db`	`Applying ... OK`
4	`.venv/bin/python manage.py migrate --database=iot`	Crea la hypertable `lectura` en `nucleo_iot`	`Applying mediciones.0001_initial... OK`
5	`.venv/bin/python manage.py sembrar_demo`	Organización de ejemplo, un usuario por rol y el dispositivo `NUC-001`	Imprime una vez la contraseña de cada usuario (guárdalas)
6	`.venv/bin/python manage.py runserver 0.0.0.0:8000`	Abre la API y el canal en vivo	`curl http://127.0.0.1:8000/api/v1/health/` devuelve `status: ok`
7	`bash Mosquitto/preparar.sh` y `docker compose up -d --wait mosquitto`	CA, certificado y usuarios del broker	Mosquitto escucha en 8883 con TLS
8	`.venv/bin/python manage.py escuchar_mqtt` (otra terminal)	Recibe lecturas y genera alertas	`Conectado al broker MQTT`
9	`cd FrontEnd && npm ci && npm run dev`	Levanta la web	`http://localhost:5173` con el menú de módulos
10	`cd Mobile && npm ci && npm start`	Levanta Expo	Código QR; la pantalla Inicio con los módulos
11	Ver la sección 5.7 (firmware) y `IoT/README.md`	Graba MicroPython y sube el código al ESP32	La pantalla OLED muestra «Calentando»

Usuarios de demostración: `plataforma@nucleo.local` (ADMINPLAT), `admin@nucleo.local`, `supervisor@nucleo.local`, `analista@nucleo.local` y `operador@nucleo.local`. Las contraseñas se generan en cada ejecución de `sembrar_demo`.

3.4 Problemas frecuentes

Síntoma	Causa probable	Solución
Django no conecta a PostgreSQL	Puerto o contraseña distintos a los del `.env` raíz	Comprueba `docker compose ps` y que `DATABASE_URL` use el puerto 55432 y la misma contraseña
El WebSocket se cierra con 4401	Token falso o caducado, o cuenta del administrador de plataforma	Inicia sesión de nuevo; ese rol no recibe eventos
El canal en vivo no llega entre procesos	`REDIS_URL` vacío (capa en memoria) o Redis apagado	Define `REDIS_URL` y enciende `redis`
La app móvil no alcanza la API	Usa `localhost` en un teléfono o falta el host permitido	Usa la IP local, `runserver 0.0.0.0:8000` y agrégala a `DJANGO_ALLOWED_HOSTS`
La web no puede llamar a la API	Origen no permitido por CORS	Mantén Vite en 5173 o ajusta `CORS_ALLOWED_ORIGINS`
El ESP32 no valida el certificado	`MQTT_HOST` no está entre los nombres del certificado, o la hora no está sincronizada	Define `MQTT_SERVER_NAMES`, ejecuta `preparar.sh` y verifica el NTP
Las pruebas fallan al crear la base	El usuario de la base no puede crear bases	Ejecútalas con el superusuario del contenedor (desarrollo)
El ESP32 no aparece en USB	Cable sin datos, driver o permisos	Usa un cable de datos, instala el driver CP210x/CH340 y agrega tu usuario a `dialout`

4. Mapa del proyecto: carpetas y archivos

Cada tabla explica los archivos de una carpeta. No se listan los que se generan o se instalan y no se versionan (`.venv/`, `node_modules/`, `dist/`, `__pycache__/`, `.env`, `secrets.py`, certificados, `media/`). Los `__init__.py` solo marcan paquetes de Python.

4.1 Raíz del proyecto (`Nucleo/`)

Archivos de la carpeta principal y las carpetas de primer nivel.

Archivo o carpeta	Qué contiene o para qué sirve
`BackEnd/`	API Django, ingesta MQTT, canal en vivo, alertas, reportes y pruebas.
`FrontEnd/`	Web React + Vite: esqueleto de módulos enlazados por rutas y menú.
`Mobile/`	App Expo + React Native: esqueleto de módulos enlazados por navegación.
`IoT/`	Firmware MicroPython del ESP32, bibliotecas, herramientas de grabado y pruebas de lógica.
`Mosquitto/`	Configuración del broker MQTT, ACL y scripts de certificados y usuarios de dispositivos.
`Database/`	Scripts SQL de práctica (una carpeta por base) y `crear-roles.sh`.
`Context/`	Documentación técnica completa para el equipo y asistentes de IA.
`Docs/`	SRS, modelo relacional, este informe y futuros reportes formales.
`docker-compose.yml`	Servicios `postgres`, `timescaledb`, `redis` y `mosquitto`; puertos solo en 127.0.0.1 (el broker se abre con `MQTT_BIND`).
`.env.example`	Plantilla de la configuración raíz: contraseñas de PostgreSQL, TimescaleDB, Redis y MQTT, puertos y `MQTT_BIND`/`MQTT_SERVER_NAMES`. El `.env` real no se versiona.
`.gitignore`	Excluye `.env`, `secrets.py`, certificados, `.venv`, `node_modules`, `dist`, cachés y `passwd` del broker.
`README.md`	Visión general, comandos de arranque, servicios y seguridad.

4.2 `BackEnd/` (raíz del backend)

Proyecto Django. El entorno `.venv/` no se versiona.

Archivo o carpeta	Qué contiene o para qué sirve
`manage.py`	Comando principal de Django (usa `config.settings.development`).
`requirements.txt`	Dependencias fijadas: Django 5.2.18, DRF 3.18.3, CORS, environ, psycopg 3, Channels 4.3, channels-redis, redis 6.4, Daphne, paho-mqtt, ReportLab.
`requirements-dev.txt`	Añade Ruff para análisis estático.
`pyproject.toml`	Reglas de Ruff (estilo y orden de importaciones).
`.env.example`	Plantilla de `BackEnd/.env`: claves, `DATABASE_URL`, `IOT_DATABASE_URL`, `REDIS_URL`, MQTT, correo, monitoreo, token, push e IA.
`.gitignore`	Ignora `.env`, `.venv`, `media/` y cachés.
`README.md`	Guía en lenguaje simple del backend y sus comandos.
`scripts/`	Carpeta reservada para tareas operativas (hoy solo `.gitkeep`).
`config/`	Configuración del proyecto (ver tabla siguiente).
`common/`	Utilidades compartidas por todas las apps (ver tabla).
`apps/`	Una app por tema del negocio (ver tablas).

4.3 `BackEnd/config/`

Configuración del proyecto Django.

Archivo o carpeta	Qué contiene o para qué sirve
`settings/base.py`	Ajustes comunes: apps instaladas (Daphne primero), dos bases (`default` e `iot`) con `LecturaRouter`, DRF con `TokenCaducable`, paginación de 50, límite de 10 inicios de sesión por minuto, CORS, MQTT, `CHANNEL_LAYERS` (Redis o memoria), `NUCLEO_*` (token, sin conexión, batería, push).
`settings/development.py`	DEBUG, CORS para Vite en 5173 y renderer navegable de DRF.
`settings/production.py`	Exige `DJANGO_ALLOWED_HOSTS`, redirección SSL, cookies seguras y HSTS/proxy configurables.

Archivo o carpeta	Qué contiene o para qué sirve
`urls.py`	Rutas raíz: `/admin/` y `/api/v1/`.
`asgi.py`	`ProtocolTypeRouter`: HTTP de Django y WebSocket de `apps.tiempo_real`.
`wsgi.py`	Entrada WSGI (alternativa sin WebSocket).

4.4 BackEnd/common/

Código transversal; ninguna app de negocio depende de otra de forma circular gracias a esta capa.

Archivo o carpeta	Qué contiene o para qué sirve
`roles.py`	Constantes de roles (`ADMINPLAT`, `ADMIN`, `SUPER`, `ANALISTA`, `OPER`) y de los catálogos de estado y variable.
`permissions.py`	`PermisoPorRol`: cada vista declara `roles_lectura`, `roles_escritura` y `roles_acciones`.
`scoping.py`	Aislamiento de datos: toda consulta parte de la organización; el operador ve su dispositivo y supervisor/analista con área, su área.
`autenticacion.py`	`TokenCaducable`: token DRF que caduca a los 7 días sin uso y se renueva con la actividad; `usuario_de_token` lo reutiliza el WebSocket.
`exceptions.py`	Formato uniforme de errores de la API.
`filtros.py`	Lectura de parámetros de consulta comunes (enteros, fechas, booleanos).
`serializers.py`	Mixin para valores por defecto que vienen de la base (`db_default`).
`testing.py`	`PruebaBase`, `PruebaTransaccional` (Channels), `Escenario` y `cliente_de` para las pruebas.

4.5 apps/core/

Salud de la API y enrutado entre bases.

Archivo o carpeta	Qué contiene o para qué sirve
`views.py`	Vista pública `GET /api/v1/health/`.
`urls.py`	Compone las rutas de todas las apps bajo `/api/v1/`.
`routers.py`	`LecturaRouter`: manda la app `mediciones` a la base `iot` y limita sus migraciones a ella.
`tests.py`	Pruebas del endpoint de salud.

4.6 apps/catalogos/

Tablas de catálogo cargadas por migración.

Archivo o carpeta	Qué contiene o para qué sirve
`models.py`	`rol`, `estado_usuario`, `estado_dispositivo`, `estado_sensor`, `variable_ambiental` (con su rango físico).
`migrations/0001_initial.py`	Crea las cinco tablas.
`migrations/0002_datos_iniciales.py`	Carga roles, estados y las variables TEMP, HUM, AIRE.
`serializers.py`, `views.py`, `urls.py`	`GET /catalogos/`: roles, estados y variables.
`admin.py`, `apps.py`, `tests.py`	Panel de administración, configuración y pruebas.

4.7 apps/organizaciones/

Organizaciones clientes y sus áreas.

Archivo o carpeta	Qué contiene o para qué sirve
`models.py`	`organizacion` (estado ACTIVA/SUSPENDIDA/BAJA) y `area`.
`serializers.py`	Solo el administrador de plataforma cambia el estado; las áreas no repiten nombre.
`views.py`	`/organizaciones/`, `/areas/` y `POST /organizaciones/{num}/primer-administrador/`.
`urls.py`, `admin.py`, `apps.py`	Rutas, administración y configuración.
`migrations/0001_initial.py`	Crea ambas tablas.

4.8 apps/usuarios/

Usuarios, sesión y teléfonos para push.

Archivo o carpeta	Qué contiene o para qué sirve
models.py	Usuario (modelo de Django, acceso por correo, permisos desde el rol) y TelefonoPush.
serializers.py	Usuario, login, cierre de sesión, primer administrador y cambio de contraseña.
views.py	auth/login, auth/logout, auth/yo, auth/cambiar-password y /usuarios/; registra y elimina el teléfono Expo; límite 10/min.
management/commands/sembrar_demo.py	Crea organización, área, un usuario por rol y el dispositivo NUC-001; imprime las contraseñas una vez (solo con DEBUG).
urls.py, admin.py, apps.py, tests.py	Rutas, administración, configuración y pruebas (incluye caducidad del token).
migrations/0001_initial.py	Crea usuario y telefono_push.

4.9 apps/dispositivos/

Nodos ESP32 y sus sensores.

Archivo o carpeta	Qué contiene o para qué sirve
models.py	dispositivo (código MQTT, intervalo, batería, últimas conexión y arranque) y sensor.
serializers.py	Dispositivo con sus sensores anidados; sensor.
views.py	/dispositivos/ (alta y baja: ADMIN), ultimas-lecturas/ y /sensores/; publica la configuración y emite el estado en vivo.
services.py	Reglas de negocio: marcar "Sin conexión", emitir el cambio de estado y publicar la configuración al confirmarse la transacción.
management/commands/marcar_sin_conexion.py	Marca dispositivos sin datos por más de 3 intervalos (--cada N para repetir).
urls.py, admin.py, apps.py, tests.py	Rutas, administración, configuración y pruebas.
migrations/0001_initial.py	Crea ambas tablas.

4.10 apps/umbrales/

Límites de advertencia y peligro.

Archivo o carpeta	Qué contiene o para qué sirve
models.py	umbral: cuatro límites por variable y dispositivo, con CHECK de orden y clasificar().
serializers.py, views.py, urls.py	/umbrales/ (escritura ADMIN y SUPER, filtro ?dispositivo=); publica la configuración al dispositivo.
admin.py, apps.py, tests.py	Administración, configuración y pruebas.
migrations/0001_initial.py	Crea la tabla.

4.11 apps/mediciones/

Lecturas (TimescaleDB) e ingesta MQTT.

Archivo o carpeta	Qué contiene o para qué sirve
models.py	Lectura en la hypertable: valor, fecha_hora, recibida_en, estabilizada; CHECK de variable y rango físico.
migrations/0001_initial.py	Crea la tabla y, con RunSQL, la llave (num, fecha_hora), identidad GENERATED ALWAYS y create_hypertable por día.
services.py	Valida el mensaje JSON, guarda lecturas, actualiza el dispositivo (conexión, batería, estado) y dispara alertas y eventos.
mqtt.py	ClienteMQTT (Singleton con TLS y reconexión) y publicar_configuracion (mensaje retenido).
consultas.py	Une las dos bases a nivel de código: resuelve sensores en nucleo_db y consulta lecturas en nucleo_iot.
serializers.py, views.py, urls.py	/lecturas/ y /lecturas/serie/ (minuto, hora, día).

Archivo o carpeta	Qué contiene o para qué sirve
`management/commands/escuchar_mqtt.py`	Proceso que escucha el broker y llama a `services`.
`apps.py`, `tests.py`	Configuración y pruebas de ingesta, series y restricciones.

4.12 `apps/alertas/`

Alertas, notificaciones, correo y push.

Archivo o carpeta	Qué contiene o para qué sirve
`models.py`	`alerta` (copia número, fecha y valor de la lectura) y `notificacion` (canal, estado de envío).
`services.py`	`generar_alerta`: una pendiente por umbral y nivel; crea notificaciones y emite el evento en vivo.
`notificaciones.py`	Correo (supervisores del área o administradores) y push con la API de Expo (apagado por defecto; borra teléfonos `DeviceNotRegistered`).
`serializers.py`, `views.py`, `urls.py`	`/alertas/` (filtros), `atender/`, `descartar/` y `/notificaciones/`.
`admin.py`, `apps.py`, `tests.py`	Administración, configuración y pruebas (incluye `PushExpoTests`).
`migrations/0001_initial.py`	Crea ambas tablas.

4.13 `apps/reportes/`

Reportes, programaciones, IA y PDF.

Archivo o carpeta	Qué contiene o para qué sirve
`models.py`	`reporte` (cifras JSON, contenido, PDF) y `programacion_reporte`.
`servicios.py`	`calcular_cifras`: promedios, mínimos, máximos, picos, alertas por estado/nivel/tipo/variable y `series` por hora o día (solo lecturas estabilizadas).
`ia.py`	Pide a un proveedor de IA solo la redacción; sin proveedor responde 503.
`pdf.py`	PDF con ReportLab: cifras, curva por variable, picos y conteo por nivel.
`serializers.py`, `views.py`, `urls.py`	`/reportes/`, `/reportes/{num}/pdf/` y `/programaciones-reporte/`.
`management/commands/generar_reportes_programados.py`	Genera y envía por correo los reportes vencidos; se programa con cron.
`admin.py`, `apps.py`, `tests.py`	Administración, configuración y pruebas.
`migrations/0001_initial.py`	Crea ambas tablas.

4.14 `apps/tiempo_real/`

Canal en vivo por WebSocket.

Archivo o carpeta	Qué contiene o para qué sirve
`consumers.py`	`PanelConsumer`: autentica con `?token=`, une al grupo `org.<n>`, filtra por rol y responde `ping` con `pong`.
`eventos.py`	Publica `alerta.nueva`, `alerta.atendida`, `alerta.descartada` y `dispositivo.estado` con `transaction.on_commit`; un fallo de Redis no interrumpe a quien publica.
`routing.py`	Ruta `ws/panel/`.
`apps.py`, `tests.py`	Configuración y pruebas de conexión, rechazo 4401, filtros por rol y eventos.

4.15 `FrontEnd/` (web React + Vite)

Esqueleto: cada módulo muestra «Hola mundo, este es el módulo de…». `node_modules/` y `dist/` no se versionan.

Archivo o carpeta	Qué contiene o para qué sirve
`package.json` / `package-lock.json`	React 19.3, React Router 7, Vite 8; scripts `dev`, `build`, `lint`, `preview`. El lock fija las versiones.
`vite.config.js`	Plugin de React; puerto 5173 con `strictPort` (CORS del backend lo espera).
`index.html`	Página que abre el navegador; monta `src/main.jsx`.
`.env.example`	`VITE_API_BASE_URL=http://localhost:8000/api/v1`.

Archivo o carpeta	Qué contiene o para qué sirve
`.oxlintrc.json`	Reglas de Oxlint para React.
`public/favicon.svg`	Icono.
`src/main.jsx`	Enciende React con `BrowserRouter`.
`src/App.jsx`	Una `Route` por cada módulo de `features/index.js`, más `/` (inicio).
`src/layouts/MainLayout.jsx`	Encabezado, menú de módulos (`NavLink`) y zona de contenido.
`src/features/index.js`	**Registro de módulos**: `{ruta, titulo, Pagina}`. Agregar un módulo = su carpeta + una línea aquí.
`src/features/<modulo>/<Modulo>Page.jsx`	Ocho módulos: `acceso`, `panel`, `dispositivos`, `mediciones`, `alertas`, `reportes`, `usuarios`, `organizaciones`. Cada uno usa `ModuloBase`.
`src/components/ModuloBase.jsx`	Contenido provisional común: título, saludo, descripción y carpeta de trabajo.
`src/components/HealthStatusCard.jsx`	Tarjeta del estado de la API.
`src/pages/HomePage.jsx`	Inicio: estado de la API y endpoint configurado.
`src/hooks/useHealthCheck.js`	Consulta `/health/`, cancela con `AbortController` y permite reintentar.
`src/services/api/client.js`	URL base, `fetch`, errores (`ApiError`). Punto único para añadir el token cuando exista la sesión.
`src/services/api/health.js`	Llamada a `GET /health/`.
`src/utils/formatDate.js`	Formato de fecha y hora.
`src/styles/global.css`	Variables de color (modo claro/oscuro del sistema), layout, menú y tarjetas.
`src/assets/`	Recursos locales (vacío).

4.16 `Mobile/` (app Expo + React Native)

Esqueleto: cada módulo muestra «Hola mundo, este es el módulo de…». No hay carpetas nativas versionadas.

Archivo o carpeta	Qué contiene o para qué sirve
`package.json` / `package-lock.json`	Expo 58.0.19, React Native 0.88, React Navigation (nativa y pila), screens y safe-area-context.
`app.json`	Nombre, `slug`, esquema `nucleo` e identificadores provisionales `com.nucleo.mobile`.
`index.js`	Registra la app en Expo.
`App.js`	Envuelve en `SafeAreaProvider` y monta `AppNavigator`.
`.env.example`	`EXPO_PUBLIC_API_BASE_URL` (10.0.2.2 en emulador Android, IP local en teléfono).
`.oxlintrc.json`	Reglas de Oxlint.
`src/navigation/AppNavigator.js`	Pila de pantallas: `Inicio` y una por cada módulo de `modules/index.js`.
`src/modules/index.js`	**Registro de módulos**: `{nombre, titulo, Pantalla}`. Agregar un módulo = su carpeta + una línea aquí.
`src/modules/<modulo>/<Modulo>Screen.js`	Cinco módulos: `acceso`, `panel`, `dispositivos`, `mediciones`, `alertas`. Cada uno usa `ModuloBase`.
`src/screens/HomeScreen.js`	Inicio: estado de la API, botón por módulo y endpoint configurado.
`src/components/ModuloBase.js`	Contenido provisional común.
`src/components/HealthStatusCard.js`	Tarjeta del estado de la API.
`src/hooks/useHealthCheck.js`	Consulta `/health/` y permite reintentar.
`src/services/api/client.js`, `health.js`	Cliente HTTP de la API (misma interfaz que la web) y llamada de salud.
`src/utils/formatDate.js`	Formato de fecha y hora.
`src/theme.js`	Colores y radios.

4.17 `IoT/` (ESP32 con MicroPython)

Código del dispositivo, herramientas de la computadora y pruebas.

Archivo o carpeta	Qué contiene o para qué sirve
`requirements.txt`	`esptool` 5.6, `mpremote` 1.29 y los stubs de MicroPython para el editor.

Archivo o carpeta	Qué contiene o para qué sirve
`README.md`	Guía paso a paso: grabar MicroPython, credenciales, certificado, subir el código.
`micropython/boot.py`	Arranque mínimo: apaga mensajes de depuración y libera memoria.
`micropython/main.py`	Inicia `nucleo.aplicacion.ejecutar()` y reinicia la placa ante un error.
`micropython/config.py`	Pines, intervalos (30 s; 5 a 120), rangos físicos, calentamiento 20 min, baseline, cola, batería, Wi-Fi/MQTT y tópicos.
`micropython/secrets.example.py`	Plantilla: Wi-Fi, `CODIGO_MQTT` (también usuario y client ID), `MQTT_HOST` y contraseña del dispositivo. `secrets.py` no se versiona.
`micropython/pymakr.conf`	Proyecto `nucleo-esp32` para Pymakr; no sube `firmware/`.
`micropython/firmware/ESP32_GENERIC-20260824-v1.29.0.bin`	Imagen de MicroPython que se graba una vez por placa.
`micropython/nucleo/aplicacion.py`	Ciclo principal: leer, clasificar, mostrar, encolar y publicar; atiende Wi-Fi, MQTT y buzzer cada 100 ms.
`micropython/nucleo/sensores.py`	DHT22 y CCS811: rangos, reintento del CCS811, calentamiento y baseline (guardar cada hora, restaurar al iniciar).
`micropython/nucleo/umbrales.py`	Configuración recibida (se guarda en `ajustes.json`) y clasificación Normal/Advertencia/Peligro igual a la del backend.
`micropython/nucleo/cola.py`	Cola en memoria (20) de mensajes con hora para reenviar tras una caída.
`micropython/nucleo/comunicacion.py`	Cliente MQTT con TLS (valida la CA), publica lecturas y recibe la configuración retenida.
`micropython/nucleo/red.py`	Wi-Fi con reintentos y hora por NTP.
`micropython/nucleo/indicadores.py`	LED RGB, LED de Wi-Fi y buzzer.
`micropython/nucleo/pantalla.py`	Pantalla OLED SSD1306.
`micropython/nucleo/bateria.py`	Nivel de batería por ADC (desactivado hasta montar el divisor).
`micropython/lib/ccs811.py`, `ssd1306.py`, `umqtt/`, `mq135.py`	Bibliotecas copiadas a la placa. `mq135.py` no se usa (la calidad del aire la mide el CCS811).
`tests/stubs.py`	Módulos de MicroPython simulados (`machine`, `dht`, `ccs811`…) y reloj controlable.
`tests/test_logica.py`	16 pruebas: umbrales, calentamiento, baseline, cola y forma del mensaje.

4.18 `Mosquitto/` (broker MQTT)

Configuración del broker; `config/passwd` y `certs/` se generan y no se versionan.

Archivo o carpeta	Qué contiene o para qué sirve
`config/mosquitto.conf`	Un solo listener en 8883 con TLS 1.2, sin acceso anónimo, usuarios y ACL.
`config/acl`	`nucleo_backend` lee `measurements` y escribe `config`; `pattern ... %u` limita cada usuario de dispositivo a su propio tópico.
`preparar.sh`	Crea la CA y el certificado del servidor (con `MQTT_SERVER_NAMES`), y los usuarios `nucleo_backend` y `NUC-001`; idempotente, conserva la CA.
`dispositivo.sh <codigo>`	Crea o cambia el usuario MQTT de un dispositivo con contraseña aleatoria mostrada una vez y recarga el broker.

4.19 `Database/` (scripts SQL y roles)

Scripts manuales; no forman parte de la inicialización de Docker ni los usa el backend.

Archivo o carpeta	Qué contiene o para qué sirve
`README.md`	Índice de las dos bases y de `crear-roles.sh`.
`crear-roles.sh`	Crea `nucleo_backend` y `nucleo_backend_iot` sin superusuario, dueños de su base y sin acceso a la ajena. Lee `BACKEND_DB_PASSWORD` y `BACKEND_IOT_DB_PASSWORD` del `.env`.
`Web-Mobile/init.sql`, `drop.sql`	Crean/eliminan la base de práctica `nucleo_test` (PostgreSQL).
`Web-Mobile/tables.sql`	Las 16 tablas del modelo con sus restricciones.
`Web-Mobile/seed.sql`	Catálogos y datos de práctica.
`Web-Mobile/queries.sql`	Consultas de alertas, límite de 2 minutos, destinatarios y reportes.

Archivo o carpeta	Qué contiene o para qué sirve
`Web-Mobile/procedures.sql`, `triggers.sql`	Vacíos: espacio para practicar.
`Web-Mobile/README.md`	Cómo usar cada script.
`IoT/init.sql`, `drop.sql`	Crean/eliminan `nucleo_iot_test` (TimescaleDB).
`IoT/tables.sql`	Extensión, tabla `lectura` e hypertable.
`IoT/seed.sql`	Lecturas de ejemplo.
`IoT/queries.sql`	Consultas por intervalo, calentamiento, latencia y chunks.
`IoT/README.md`	Cómo usar cada script.

4.20 `Context/` (documentación técnica)

Una carpeta por tema; es la fuente de verdad detallada.

Archivo o carpeta	Qué contiene o para qué sirve
`README.md`	Índice del contexto.
`arquitectura/arquitectura-general.md`	Componentes, diagrama de despliegue, flujos, comunicación y límites abiertos.
`instalacion/guia-desarrollo-local.md`	Requisitos, `.env`, orden de inicio, verificación y problemas frecuentes.
`backend/estructura-backend.md`	Apps, variables, autenticación, ingesta, canal en vivo, reportes, pruebas y producción.
`frontend/estructura-frontend.md`	Esqueleto web, registro de módulos y pendientes.
`mobile/estructura-mobile.md`	Esqueleto móvil, registro de módulos y verificación.
`api/contrato-api.md`	Endpoints, autenticación, roles, errores y canal WebSocket.
`base-de-datos/modelo-relacional.md`	Las 16 tablas y la hypertable.
`base-de-datos/postgresql-docker.md`	Servicios Docker, usuarios sin superusuario, bases de práctica y respaldos.
`mqtt/broker-mosquitto.md`	Tópicos, mensajes, usuarios, ACL y alta de dispositivos.
`iot/firmware-y-hardware.md`	Hardware, pines, bibliotecas, funcionamiento y calibración.
`decisiones-tecnicas/stack-tecnologico.md`	Decisiones aceptadas, provisionales y pendientes.
`pruebas/README.md`	Cobertura, comandos y pendientes de pruebas.
`reportes/verificacion-inicial.md`	Línea base de la preparación del entorno.

4.21 `Docs/` (documentos del proyecto)

Documentos formales; los reportes de avance se agregan aquí.

Archivo o carpeta	Qué contiene o para qué sirve
`SRS.pdf`	Especificación de requisitos de software: fuente de los requisitos.
`Modelo relacional y requerimientos.pdf`	Modelo relacional y requerimientos que originaron las 16 tablas.
`informe-2026-10-24-nucleo.pdf`	Este informe: avance, mapa de carpetas, guía de integración, uso del repositorio y pendientes del SRS.

5. Cómo funciona por dentro

Detalles que no caben en un README: el modelo, los permisos, los contratos y las reglas que conviene conocer antes de tocar el código.

5.1 Modelo de datos

Las migraciones de Django crean las tablas con los nombres del modelo relacional (`db_table` y `db_column`). Las reglas viven en la base con `CHECK`, `UNIQUE`, índices y `db_default`, de modo que un dato inválido se rechaza aunque falle la aplicación.

Tabla	Base	Qué guarda	Reglas importantes
`rol`, `estado_usuario`, `estado_dispositivo`, `estado_sensor`	nucleo_db	Catálogos con clave de texto (`ADMIN`, `LINEA`, `OPERA`…)	Cargados por la migración `0002`; los estados de dispositivo son `LINEA`, `CALENT`, `SINCONEX`, `BATBAJA`, `MANT` y `BAJA`
`variable_ambiental`	nucleo_db	TEMP, HUM y AIRE con unidad y rango físico	TEMP −40 a 80, HUM 0 a 100, AIRE 0 a 10000
`organizacion`	nucleo_db	Cliente de la plataforma y su estado	ACTIVA, SUSPENDIDA o BAJA; una cuenta de organización no activa no inicia sesión
`area`	nucleo_db	Subdivisión de una organización	Nombre único dentro de la organización
`usuario`	nucleo_db	Persona con acceso; correo como identificador	`CHECK`: el administrador de plataforma no tiene organización y el resto sí; supervisor y operador requieren área
`telefono_push`	nucleo_db	Token de Expo y plataforma (ANDROID o IOS)	Un usuario puede tener varios; un token que cambia de usuario se reasigna
`dispositivo`	nucleo_db	Nodo ESP32: código MQTT único, área, operador, intervalo, batería, última conexión	Intervalo con `CHECK`; el alta y la baja son del ADMIN
`sensor`	nucleo_db	Sensor de un dispositivo (modelo, variable, estado de calibración)	El mensaje MQTT usa el `modelo` del sensor
`umbral`	nucleo_db	Cuatro límites por variable y dispositivo	`CHECK`: peligro_min ≤ advertencia_min < advertencia_max ≤ peligro_max; uno por variable y dispositivo
`alerta`	nucleo_db	Lectura fuera de límites, con nivel, tipo y estado	Copia número, fecha y valor de la lectura; una PENDIENTE por umbral y nivel; ADVERTENCIA o PELIGRO; POR_ENCIMA o POR_DEBAJO
`notificacion`	nucleo_db	Aviso por usuario y canal (correo o push)	Estado de envío ENVIADA o FALLIDA
`reporte`	nucleo_db	Cifras del periodo (JSON), texto de IA y PDF	El periodo inicia antes de terminar
`programacion_reporte`	nucleo_db	Reporte periódico (diario, semanal o mensual) y destinatarios	Destinatarios de la misma organización
`lectura`	nucleo_iot	Cada medición con `fecha_hora`, `recibida_en` y `estabilizada`	Hypertable por día; llave `(num, fecha_hora)`; `CHECK` de variable y rango físico

Las dos bases no tienen llaves foráneas entre sí: `lectura` guarda `sensor_num` y `variable_codigo` como valores y `apps/mediciones/consultas.py` junta ambos lados. Django no crea reglas `ON DELETE CASCADE` ni `SET NULL`; esas reglas están solo en los scripts SQL de práctica.

5.2 Roles y alcance de los datos

Rol	Ve	Puede hacer
`ADMINPLAT` (equipo Núcleo)	Solo organizaciones; recibe 403 en el resto	Alta, suspensión y baja de organizaciones; crea el primer administrador de cada una
`ADMIN`	Toda su organización	Áreas, usuarios (no ADMINPLAT), dispositivos (alta y baja), sensores, umbrales, alertas, reportes y programaciones
`SUPER`	Su área (si tiene una)	Operadores de su área, intervalo de muestreo, umbrales, atender o descartar alertas, reportes
`ANALISTA`	Su área (si tiene una)	Consulta histórico y genera reportes; no atiende alertas
`OPER`	Solo su dispositivo asignado	Consulta en tiempo real; no escribe

Un recurso fuera del alcance responde 404; una acción no permitida para el rol responde 403; sin credenciales, 401. Un ADMIN no puede crear ni ascender usuarios a ADMINPLAT (400).

5.3 API REST (prefijo `/api/v1/`)

Grupo	Rutas	Quién escribe	Notas
Salud y catálogos	`GET /health/`, `GET /catalogos/`	—	`health` es público
Sesión	`POST /auth/login/`, `/auth/logout/`, `GET /auth/yo/`, `POST /auth/cambiar-password/`	Todos	10 inicios de sesión por minuto; el móvil envía `token_expo` y `plataforma`
Organizaciones	`/organizaciones/`, `POST /organizaciones/{num}/primer-administrador/`, `/areas/`	ADMINPLAT; ADMIN en lo propio	409 si la organización ya tiene administrador
Usuarios	`/usuarios/`	ADMIN y SUPER	La baja marca la cuenta Inactiva
Dispositivos	`/dispositivos/`, `/dispositivos/{num}/ultimas-lecturas/`, `/sensores/`	ADMIN (alta y baja); ADMIN y SUPER (intervalo)	Estados: LINEA, CALENT, SINCONEX, BATBAJA, MANT, BAJA
Umbrales	`/umbrales/?dispositivo=`	ADMIN y SUPER	Cambiarlos publica la configuración al dispositivo
Mediciones	`GET /lecturas/`, `GET /lecturas/serie/`	Solo lectura	La serie exige `dispositivo` y `variable`; `intervalo` = minuto, hora o día; sin fechas, últimas 24 h; solo lecturas estabilizadas
Alertas	`/alertas/`, `POST /alertas/{num}/atender/`, `.../descartar/`, `/notificaciones/`	ADMIN y SUPER	409 si la alerta ya se resolvió
Reportes	`/reportes/`, `GET /reportes/{num}/pdf/`, `/programaciones-reporte/`	ADMIN, SUPER y ANALISTA	503 si el servicio de IA no está disponible

Las listas están paginadas (`count`, `next`, `previous`, `results`; 50 por página). Los errores 400 llegan como `{"campo": ["mensaje"]}`. Las fechas son ISO 8601 con zona horaria. El contrato completo está en `Context/api/contrato-api.md`.

5.4 Token y sesión

- El inicio de sesión devuelve un token de DRF; se envía como `Authorization: Token <token>` (la web y el móvil usan el mismo mecanismo).
- El token caduca a los **7 días sin uso** (`NUCLEO_TOKEN_DIAS_SIN_USO`). La fecha de creación del token se usa como «último uso»: cada petición la renueva, como máximo una vez por minuto para no escribir en cada llamada.
- Con un token caducado la API responde 401; un nuevo inicio de sesión crea un token nuevo. El token también deja de valer al cerrar sesión o al cambiar la contraseña.
- Solo inician sesión las cuentas Activas de una organización activa.
- La app móvil, al iniciar sesión, registra su token de Expo; `POST /auth/logout/` con `token_expo` lo elimina.

5.5 Canal en vivo (WebSocket)

```
ws://<host>:8000/ws/panel/?token=<token de la API>
```

Evento	Cuándo se emite	Datos principales
`conectado`	Al aceptar la conexión	`usuario`
`alerta.nueva`	Se crea una alerta	`num`, `estado`, `nivel`, `tipo`, `variable`, `dispositivo`, `dispositivo_nombre`, `valor`, `fecha_hora`
`alerta.atendida`, `alerta.descartada`	Alguien resuelve la alerta	Los mismos campos
`dispositivo.estado`	Cambia el estado del dispositivo (en línea, calentando, sin conexión…)	`dispositivo`, `nombre`, `estado`, `nivel_bateria`, `ultima_conexion`
`ping` / `pong`	El cliente envía `{"tipo": "ping"}`	Mantiene viva la conexión

- Rechazo con el código **4401** si el token falta, es falso o está caducado, o si la cuenta es del administrador de plataforma.

- Filtro por rol: el operador recibe los eventos de su dispositivo; supervisor y analista con área, los de su área; el administrador, todos. La caducidad se revisa al conectar.
- Los eventos se publican con `transaction.on_commit` y no interrumpen al llamador si Redis falla. No se transmiten lecturas: para ellas se consulta `ultimas-lecturas`.
- El cliente debe reconectar con espera creciente y volver a pedir por la API lo que haya cambiado mientras estuvo desconectado.
- Detalle técnico: `redis-py` 6.4 y `channels-redis` 4.3 fijados en `requirements.txt` (con una versión más nueva de `redis-py`, las conexiones WebSocket inactivas se cerraban con un error de tiempo de espera).

5.6 MQTT y Mosquitto

Tópico	Quién publica	Quién lee	Contenido
`nucleo/devices/<codigo_mqtt>/measurements`	El dispositivo	El backend (`nucleo_backend`)	Lecturas, batería, hora y marca `aire_estabilizado`
`nucleo/devices/<codigo_mqtt>/config`	El backend (mensaje retenido)	El dispositivo	Intervalo de muestreo y umbrales vigentes

```
{
  "dispositivo": "NUC-001",
  "aire_estabilizado": true,
  "fecha_hora": "2026-10-24T15:30:00Z",
  "bateria": 87,
  "lecturas": [
    {"sensor": "DHT22",  "variable": "TEMP", "valor": 23.4},
    {"sensor": "DHT22",  "variable": "HUM",  "valor": 48.0},
    {"sensor": "CCS811", "variable": "AIRE", "valor": 412}
  ]
}
```

- `fecha_hora`, `bateria` y `aire_estabilizado` son opcionales; sin hora se usa la de recepción. `aire_estabilizado: false` guarda el aire con `estabilizada = false`, deja el dispositivo en `CALENT` y no genera alertas por él.
- Seguridad: un solo listener TLS en 8883, sin acceso anónimo. Cada dispositivo tiene su propio usuario (su `codigo_mqtt`), y la ACL usa `pattern ... %u`: se liga al usuario autenticado, no al client ID que el cliente elige. Un dispositivo no puede publicar en el tópico de otro.
- Altas: `bash Mosquitto/dispositivo.sh NUC-002` crea el usuario con una contraseña aleatoria mostrada una vez. Para que el ESP32 llegue al broker, `MQTT_BIND=0.0.0.0` y `MQTT_SERVER_NAMES=<ip o nombre>`, luego `preparar.sh` (reemite el certificado y conserva la CA) y `docker compose up -d mosquitto`.

5.7 Firmware del ESP32

Pieza	Pin / dirección	Notas
DHT22 (temperatura y humedad)	GPIO4	Pull-up; mínimo 2 s entre lecturas
CCS811 (eCO2 y TVOC) e I2C	SDA GPIO21, SCL GPIO22, dirección 0x5A	nWAKE a GND; 48 h de acondicionamiento inicial y 20 min tras cada arranque
Pantalla OLED SSD1306 128×64	I2C 0x3C (mismo bus)	Muestra valores, estado y «Calentando»
LED RGB	GPIO25 rojo, GPIO26 verde, GPIO27 azul	Verde normal, amarillo advertencia, rojo peligro
LED de Wi-Fi	GPIO33	Encendido con la conexión
Buzzer activo	GPIO32	Suena a pulsos en peligro
Batería Li-ion	GPIO35 (ADC1)	Divisor 1:2; medición desactivada (`BATERIA_CONECTADA = False`) hasta montarlo

Los pines son provisionales y están en `IoT/micropython/config.py`. Se usa ADC1 porque ADC2 no funciona con el Wi-Fi activo.

- **Ciclo:** cada intervalo (30 s por defecto, de 5 a 120) lee, clasifica, actualiza LED, pantalla y buzzer, y publica; entre muestras atiende Wi-Fi, MQTT y buzzer cada 100 ms. Un sensor que falla no impide leer ni publicar los demás; las lecturas fuera del rango físico no se publican.
- **Calentamiento:** durante 20 minutos el mensaje lleva `aire_estabilizado: false` y el aire no activa LED ni buzzer; temperatura y humedad alertan desde el arranque.

- **Sin conexión:** sigue alertando localmente con la última configuración (`ajustes.json`). Los mensajes con hora se guardan en una cola en memoria (20) y se reenvían del más antiguo al más nuevo, 5 por ciclo; sin hora no se encolan, porque la plataforma los fecharía al recibirlos. La cola se pierde si la placa se reinicia.
- **Baseline del CCS811:** ya estabilizado, se guarda en `baseline.json` cada hora y se restaura al arrancar.
- **Hora y TLS:** NTP al conectar el Wi-Fi; la hora es necesaria para validar el certificado, que siempre se valida contra la CA (`certs/ca.der`).
- **Cargar el código:** grabar MicroPython una vez (`esptool`, offset `0x1000`); `cp secrets.example.py secrets.py`; convertir la CA a DER; subir `lib certs nucleo config.py secrets.py boot.py main.py` con Pymakr o `mpremote`. Paso a paso en `IoT/README.md`.

5.8 Reportes

- `calcular_cifras` agrega en TimescaleDB solo lecturas estabilizadas: promedio, mínimo, máximo y picos por variable, y conteo de alertas por estado, nivel, tipo y variable. También genera `series` por variable (promedio, mínimo y máximo por tramo) con `intervalo_serie`: por hora si el periodo dura hasta 3 días y por día si es mayor.
- La IA solo redacta el texto a partir de esas cifras. Si no hay proveedor configurado o no responde, la API responde 503 y no guarda nada.
- El PDF (ReportLab) incluye cifras, una curva por variable, picos y conteo por nivel; se guarda en `media/reportes/` y se descarga con `GET /reportes/{num}/pdf/` (requiere token).
- Las programaciones diarias, semanales o mensuales las ejecuta `manage.py generar_reportes_programados` (cron): genera el reporte del periodo anterior, lo envía por correo con el PDF adjunto y fija la siguiente ejecución; si la IA no responde, reintenta en la siguiente corrida.

5.9 Correo y notificaciones push

- Cada alerta crea una notificación por responsable y canal. El correo va a los supervisores del área; si no hay, a los administradores. En desarrollo el correo se imprime en consola.
- El push usa la API de Expo (`NUCLEO_EXPO_PUSH_URL`) y está **apagado** (`NUCLEO_PUSH_HABILITADO=False`). Si Expo responde `DeviceNotRegistered`, se elimina ese teléfono. Un fallo nunca interrumpe la ingesta.

5.10 Web y app móvil: cómo repartir el trabajo

Ambos clientes son un esqueleto con el mismo patrón: un **registro de módulos** genera las rutas (web) o las pantallas (móvil) y los accesos de navegación. Cada módulo es una carpeta independiente que se puede trabajar en paralelo sin tocar a los demás.

Módulo	Web (`FrontEnd/src/features/`)	Móvil (`Mobile/src/modules/`)	Endpoints que usará
Acceso	`acceso/AccesoPage.jsx`	`acceso/AccesoScreen.js`	`auth/login`, `auth/logout`, `auth/yo`
Panel	`panel/PanelPage.jsx`	`panel/PanelScreen.js`	`dispositivos`, `ultimas-lecturas`, `alertas`, WebSocket
Dispositivos	`dispositivos/DispositivosPage.jsx`	`dispositivos/DispositivosScreen.js`	`dispositivos`, `sensores`
Mediciones	`mediciones/MedicionesPage.jsx`	`mediciones/MedicionesScreen.js`	`lecturas`, `lecturas/serie`
Alertas	`alertas/AlertasPage.jsx`	`alertas/AlertasScreen.js`	`alertas`, `atender`, `descartar`, `notificaciones`
Reportes	`reportes/ReportesPage.jsx`	(solo web)	`reportes`, `reportes/{num}/pdf`, `programaciones-reporte`
Usuarios	`usuarios/UsuariosPage.jsx`	(solo web)	`usuarios`, `areas`
Organizaciones	`organizaciones/OrganizacionesPage.jsx`	(solo web)	`organizaciones`, `primer-administrador`

- Para agregar un módulo en la web: carpeta en `src/features/` con su página y una línea en `src/features/index.js`. En el móvil: carpeta en `src/modules/` con su pantalla y una línea en `src/modules/index.js`.
- Al implementar la sesión, el token debe añadirse en `services/api/client.js` (único punto de salida de las peticiones) y la ruta de cada módulo debe respetar la matriz de roles de la sección 5.2.

- El SRS pide diseño minimalista con modo claro y oscuro (sigue al sistema y se puede cambiar a mano) y verde como acento; la web ya define variables de color para ambos modos en `styles/global.css`.

5.11 Decisiones y diferencias a tener presentes

Tema	Decisión o particularidad	Por qué importa
Dos bases	`nucleo_db` (PostgreSQL) y `nucleo_iot` (TimescaleDB), con `LecturaRouter` y sin llaves foráneas entre ellas	`migrate` y `migrate --database=iot` se ejecutan por separado; la alerta copia los datos de la lectura
Llave de la hypertable	La migración usa `RunSQL` para dejar `(num, fecha_hora)` como llave e identidad `GENERATED ALWAYS`	Django no lo expresa; no regenerar esa migración con `makemigrations`
`ON DELETE`	Django no crea `CASCADE` ni `SET NULL`; solo están en los scripts SQL de práctica	Si se necesitan en producción hay que agregarlas con una migración
Usuario propio	`usuarios.Usuario` sin `PermissionsMixin`; los permisos salen del rol	No hay grupos ni permisos de Django; `/admin/` solo entra ADMINPLAT
Zona horaria	`America/Tijuana` con `USE_TZ`; se guarda en UTC	Los clientes reciben ISO 8601 con zona
Token	Token de DRF con caducidad propia, no JWT	Cambiar a JWT implicaría reescribir `common/autenticacion.py` y el consumidor WebSocket
Pruebas con Channels	`PruebaTransaccional` (transacciones reales) y capa en memoria con `test`	Channels cierra conexiones; las pruebas del canal no usan `PruebaBase`
Daphne primero	`daphne` es la primera app instalada	`runserver` atiende HTTP y WebSocket; en producción se sirve con `daphne config.asgi:application`
Contenedores en desarrollo	Los usuarios de las imágenes son superusuarios dentro de su contenedor	Son credenciales de desarrollo; en producción se usan los roles de `crear-roles.sh`
Límite de 2 minutos	`NUCLEO_ALERTAS_MAX_SEGUNDOS = 120` es una constante documentada; la alerta se crea en la misma transacción de la ingesta	En la prueba extremo a extremo la alerta se creó en el mismo segundo de la recepción; no hay temporizador que lo imponga

6. Base de datos y pruebas locales

En el equipo conviven tres espacios de datos distintos. Confundirlos es la causa más común de pérdidas de datos de desarrollo, así que conviene tenerlos claros.

6.1 Los tres espacios

Espacio	Dónde	Quién lo crea	Para qué sirve	¿Se puede borrar?
Bases de desarrollo	`nucleo_db` (PostgreSQL) y `nucleo_iot` (TimescaleDB)	Las imágenes de Docker crean la base vacía; las migraciones de Django crean las tablas	Lo que usa la API, el consumidor MQTT y las pantallas; incluye los datos de `sembrar_demo`	Solo recreándolas a propósito (destruye los datos; ver 6.3)
Bases de práctica SQL	`nucleo_test` y `nucleo_iot_test`	`Database/*/init.sql` a mano	Practicar tablas, consultas, procedimientos y triggers sin tocar las reales; el backend no las usa	Sí: `drop.sql` o repetir `init.sql`
Bases temporales de Django	`test_nucleo_db` y `test_nucleo_iot`	`manage.py test` las crea y las elimina	Ejecutar las 134 pruebas aisladas de los datos reales	No hace falta: se borran solas (`--noinput`)

6.2 Comandos habituales

```
# Bases de desarrollo: tablas y catálogos
cd BackEnd
.venv/bin/python manage.py migrate
.venv/bin/python manage.py migrate --database=iot
.venv/bin/python manage.py sembrar_demo        # datos de ejemplo (solo con DEBUG)

# Pruebas automáticas (crean y borran test_nucleo_db y test_nucleo_iot)
.venv/bin/python manage.py test --noinput

# Práctica SQL: PostgreSQL (16 tablas)
docker exec -i nucleo_postgres psql -U nucleo_app -d postgres -v ON_ERROR_STOP=1 < Database/Web-Mobile/init.sql
docker exec -i nucleo_postgres psql -U nucleo_app -d nucleo_test -v ON_ERROR_STOP=1 < Database/Web-Mobile/tables.sql
docker exec -i nucleo_postgres psql -U nucleo_app -d nucleo_test -v ON_ERROR_STOP=1 < Database/Web-Mobile/seed.sql

# Práctica SQL: TimescaleDB (hypertable lectura)
docker exec -i nucleo_timescaledb psql -U nucleo_iot -d postgres -v ON_ERROR_STOP=1 < Database/IoT/init.sql
docker exec -i nucleo_timescaledb psql -U nucleo_iot -d nucleo_iot_test -v ON_ERROR_STOP=1 < Database/IoT/tables.sql
docker exec -i nucleo_timescaledb psql -U nucleo_iot -d nucleo_iot_test -v ON_ERROR_STOP=1 < Database/IoT/seed.sql
```

6.3 Recrear una base de desarrollo (destructivo)

Solo en desarrollo y con aviso al equipo: borra todos los datos de esa base. `docker compose down` sin `-v` conserva los volúmenes; **no uses** `down -v` salvo que quieras destruirlos.

```
docker exec nucleo_postgres psql -U nucleo_app -d nucleo_db -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;"
cd BackEnd && .venv/bin/python manage.py migrate
# nucleo_iot: el mismo DROP en nucleo_timescaledb y luego: manage.py migrate --database=iot
```

6.4 Usuarios de base de datos sin superusuario

- En Docker, los usuarios `nucleo_app` y `nucleo_iot` son superusuarios dentro de su contenedor: sirven para desarrollo y para las pruebas (que necesitan crear bases).
- `bash Database/crear-roles.sh` crea `nucleo_backend` y `nucleo_backend_iot`: sin superusuario, sin crear bases ni roles, dueños solo de su base y sin acceso a la otra (`CONNECT` revocado a `PUBLIC`). La extensión `timescaledb` la crea el script con el administrador. Es idempotente y se verificó con bases desechables: `migrate` y `migrate --database=iot` completos con el rol sin privilegios.
- Aún **no se ha aplicado** a `nucleo_db` ni a `nucleo_iot`: hacerlo implica cambiar `DATABASE_URL` e `IOT_DATABASE_URL`. Requiere `BACKEND_DB_PASSWORD` y `BACKEND_IOT_DB_PASSWORD` en el `.env` raíz.

6.5 Cosas a tener en cuenta en las pruebas locales

- Los scripts de `Database/` son una copia manual del modelo: si cambian los modelos de Django hay que actualizar `tables.sql` a mano. `procedures.sql` y `triggers.sql` están vacíos a propósito.
- Una lectura de prueba publicada por MQTT queda guardada en `nucleo_iot` (y puede crear una alerta en `nucleo_db`). Para repetir pruebas, atiende o borra las alertas pendientes: mientras haya una pendiente del mismo umbral y nivel no se crea otra.

- Las lecturas de los primeros 20 minutos del CCS811 se guardan sin estabilizar y no generan alertas ni entran en reportes; para probar alertas de aire en local, envía `aire_estabilizado: true`.
- Los PDF de reportes se escriben en `BackEnd/media/reportes/` (no se versiona).
- No hay una herramienta oficial para simular un dispositivo ni para generar histórico de varios días; hoy se publica a mano con `paho-mqtt` (ver sección 7.3).

7. Estado de verificación

7.1 Verificado

Qué	Cómo se verificó	Resultado
Modelo y migraciones	`migrate` en ambas bases tras un reinicio completo; `makemigrations --check`	16 tablas, 5 roles y la hypertable `lectura`; sin migraciones pendientes
Lógica del backend	134 pruebas automáticas	Todas correctas
Token con caducidad	Pruebas: caduca, se renueva (máx. 1/min), un login crea token nuevo	Correcto
Canal en vivo	Pruebas con capa en memoria y prueba real con Redis y Daphne: conexión inactiva 20 s, evento y ping/pong	Correcto tras fijar `redis==6.4.0`
Ingesta real	Publicación por TLS como `NUC-001`; el consumidor guardó 3 lecturas, creó 1 alerta de Peligro y emitió `dispositivo.estado` y `alerta.nueva` por Redis	Correcto
ACL del broker	Un dispositivo solo publica en su tópico aunque cambie su client ID	Correcto
Certificado del broker	`preparar.sh` repetido en una copia: conserva la CA y reemite el certificado	Idempotente
Roles de base de datos	`crear-roles.sh` sobre bases desechables y `migrate` con el rol sin privilegios	Correcto; sin acceso a la base ajena
Gráficas del PDF	Se generó un PDF de ejemplo y se revisó la imagen	Una curva por variable, con picos y conteo por nivel
Push con Expo	Pruebas con la API de Expo simulada (envío, `DeviceNotRegistered`, errores)	Correcto; sin teléfono real
Lógica del firmware	16 pruebas en la computadora con módulos simulados	Correcto
Web y móvil	Lint, `vite build`, `expo-doctor` 20/20 y bundle de Android	Correcto

7.2 No verificado todavía

- El firmware en un ESP32 real: pines, TLS contra el broker, hora por NTP, batería y calibración del CCS811.
- Web y app en un navegador o teléfono real (solo se compilan y se revisan con lint).
- El push de Expo con un teléfono real y credenciales FCM/APNs.
- Un proveedor real de IA y de correo.
- El despliegue (HTTPS, proxy inverso con WebSocket, procesos de fondo, Redis y broker en servidor).
- Integración continua y protección de ramas en GitHub (no están configuradas).

7.3 Cómo repetir la prueba extremo a extremo

Con `docker compose up -d --wait`, el broker preparado, `runserver` y `escuchar_mqtt` en marcha, y `MQTT_DEVICE_PASSWORD` exportada, este script (con el entorno del backend) simula al dispositivo `NUC-001` y debería crear una alerta de Peligro si la temperatura supera su límite. Para ver el evento en vivo, abre un WebSocket a `/ws/panel/?token=…` con un token de `supervisor@nucleo.local`.

```
import json, os, ssl, time
import paho.mqtt.client as mqtt

c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="NUC-001")
c.username_pw_set("NUC-001", os.environ["MQTT_DEVICE_PASSWORD"])
c.tls_set(ca_certs="Mosquitto/certs/ca.crt", tls_version=ssl.PROTOCOL_TLS_CLIENT)
c.connect("127.0.0.1", 8883); c.loop_start(); time.sleep(1)
mensaje = {"dispositivo": "NUC-001", "aire_estabilizado": True,
           "fecha_hora": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()), "bateria": 87,
           "lecturas": [{"sensor": "DHT22", "variable": "TEMP", "valor": 29.1},
                        {"sensor": "DHT22", "variable": "HUM", "valor": 48.0},
                        {"sensor": "CCS811", "variable": "AIRE", "valor": 412}]}
c.publish("nucleo/devices/NUC-001/measurements", json.dumps(mensaje), qos=1).wait_for_publish()
```

8. Repositorio y flujo de trabajo en Git

El proyecto vive en un solo repositorio (monorepo) en GitHub: `https://github.com/BrandonGlz/Nucleo.git`. Todas las áreas comparten el mismo historial; lo que las separa son las carpetas y las ramas. Esta sección explica cada parte del repositorio y cómo subir cambios sin pisar el trabajo de los demás.

8.1 Las ramas y para qué sirve cada una

Rama	Qué contiene	Quién escribe en ella	Cómo recibe cambios
`main`	Solo entregas estables; cada etapa cerrada lleva un tag	Nadie directamente	Se une desde `dev` al cerrar una etapa
`dev`	Integración de todas las áreas; siempre debe funcionar	Nadie directamente (salvo cambios que cruzan áreas, coordinados)	Pull request desde cada rama de área
`backend`	Trabajo del backend (`BackEnd/`)	El equipo de backend	Trae `dev` con `git merge dev`; sube a `dev` con pull request
`frontend`	Trabajo de la web (`FrontEnd/`)	El equipo de la web	Igual que `backend`
`mobile`	Trabajo de la app móvil (`Mobile/`)	El equipo móvil	Igual que `backend`
`iot`	Trabajo del firmware y del broker (`IoT/` y `Mosquitto/`)	El equipo de IoT	Igual que `backend`

- **Por qué una rama por área:** si dos áreas trabajan a la vez y suben cambios al mismo tiempo, no chocan, porque cada una escribe en su carpeta y en su rama. Los conflictos posibles se limitan a archivos compartidos (README, `docker-compose.yml`, `.env.example`).
- **Documentación y `Database/`:** `Context/`, `Docs/`, los README y `Database/` no tienen rama propia; viajan con la rama del área que los modifica. Un cambio que toca varias áreas se coordina en `dev`.
- **Varias personas en la misma área:** cada una usa una rama corta `feature/<tarea>` o `fix/<tarea>` que sale de la rama del área y vuelve a ella; se borra al integrarse.
- **Flujo del código:** rama de área → `dev` → `main`. El camino de vuelta es `git merge dev` en la rama de área, para no quedarse atrás.

8.2 Qué se sube y qué no

Se versiona	No se versiona	Por qué
Código fuente, `requirements*.txt`, `package.json` y `package-lock.json`	`.venv/`, `node_modules/`, `dist/`, `__pycache__/`, cachés	Se regeneran con un comando; pesan y cambian por equipo
Plantillas `.env.example`	`.env`, `BackEnd/.env`, `FrontEnd/.env`, `Mobile/.env`	Contienen contraseñas y claves
`secrets.example.py`	`secrets.py` del firmware	Contiene la red Wi-Fi y la contraseña MQTT del dispositivo
Configuración del broker (`mosquitto.conf`, `acl`) y los scripts	`Mosquitto/config/passwd`, `Mosquitto/certs/`, `*.key`, `*.crt`, `*.der`, `*.pem`	Usuarios del broker y certificados generados en cada equipo
SRS, informe y documentación (`Docs/`, `Context/`)	`BackEnd/media/` (PDF generados), respaldos `*.dump`, bases `*.db`	Datos generados o de cada entorno
Firmware de MicroPython (`.bin`, 1,8 MB)	Archivos temporales y de editores	El `.bin` se necesita para grabar la placa

Las reglas están en `.gitignore` (raíz y de cada carpeta). Antes de cada commit conviene revisar `git status` y `git diff --cached`; un secreto subido por error hay que **cambiarlo**, no basta con borrarlo del repositorio, porque queda en el historial.

8.3 Cómo subir cambios, con ejemplos

Ejemplo 1: una persona del backend sube un cambio

```
git checkout backend
git pull                                  # trae lo último de su rama
git merge dev                             # trae lo que otras áreas ya integraron
# ... trabajar, correr las pruebas (BackEnd: manage.py test y ruff check .) ...
git status                                # revisar qué cambió (sin .env ni secretos)
git add BackEnd/apps/reportes/            # agregar solo lo que corresponde
git commit -m "feat(backend): agrega serie de lecturas por hora"
git push origin backend
```

Después, en GitHub, se abre un **pull request de** `backend` **hacia** `dev`: título igual al commit, una o dos líneas de descripción y las pruebas del área pasando. Al unirse, el resto del equipo trae el cambio con `git merge dev`.

Ejemplo 2: dos personas del backend trabajan a la vez

```
git checkout backend && git pull
git checkout -b feature/agente-ia          # rama corta que sale de backend
# ... trabajar y hacer commits cortos ...
git push -u origin feature/agente-ia
# pull request hacia backend; al unirse, borrar la rama:
git branch -d feature/agente-ia
git push origin --delete feature/agente-ia
```

Ejemplo 3: un cambio que toca varias áreas o archivos compartidos

```
# Ejemplo: agregar una variable a docker-compose.yml y a .env.example
git checkout dev && git pull
git commit -am "chore: agrega REDIS_PORT a docker-compose y .env.example"
git push origin dev
# avisar al equipo para que cada rama de área ejecute: git merge dev
```

Ejemplo 4: resolver un conflicto al traer dev

```
git checkout frontend
git merge dev                              # Git avisa: CONFLICT en README.md
# abrir el archivo, elegir qué queda entre <<<<<<< y >>>>>>>, guardar
git add README.md
git commit                                 # cierra la unión con el mensaje que propone Git
git push origin frontend
```

Ejemplo 5: cerrar una etapa y marcarla en `main`

```
git checkout main && git pull
git merge dev                              # solo cuando dev está estable y probado
git tag -a etapa-1 -m "Etapa 1"            # un tag por etapa (el nombre lo define el equipo)
git push origin main --tags
```

Ejemplo 6: corregir errores

Situación	Comando	Qué hace
El último commit aún no se ha subido y el mensaje o contenido está mal	`git commit --amend`	Reescribe ese commit; no usar si ya se hizo `push`
Un commit ya subido salió mal	`git revert <hash>`	Crea un commit nuevo que lo deshace; el historial no se reescribe
Quiero descartar cambios sin commit de un archivo	`git restore ruta/archivo`	Vuelve al último commit; los cambios se pierden
Hice commit en la rama equivocada (sin subir)	`git switch rama-correcta` y `git cherry-pick <hash>`	Copia el commit a la rama correcta; después se quita de la equivocada
Quiero ver qué cambió	`git log --oneline`, `git diff`, `git show <hash>`	Historial resumido, cambios sin commit y detalle de un commit

No se usa `git push --force` sobre ramas compartidas. Si alguna vez es indispensable, se usa `--force-with-lease` y se avisa antes al equipo.

8.4 Mensajes de commit

Un commit es un solo cambio lógico con un mensaje de **una línea**: `tipo(alcance): resumen`, en español, con el verbo en presente, sin punto final y de máximo 72 caracteres. El cuerpo solo se escribe si el porqué no es evidente. No se agregan coautores ni etiquetas automáticas de herramientas; el commit va a nombre de quien lo hizo.

Tipo	Cuándo usarlo	Ejemplo
`feat`	Funcionalidad nueva	`feat(backend): agrega serie de lecturas por hora`
`fix`	Corrección de un error	`fix(iot): reenvía las lecturas encoladas al reconectar`
`docs`	Solo documentación	`docs: actualiza la guía de instalación`
`test`	Pruebas nuevas o ajustadas	`test(backend): cubre la caducidad del token`
`refactor`	Reorganiza sin cambiar el comportamiento	`refactor(web): mueve el cliente HTTP a services/api`
`perf`	Mejora de rendimiento	`perf(backend): indexa la consulta de alertas`
`style`	Formato sin cambiar el código	`style(backend): aplica ruff format`
`chore`	Dependencias, configuración y tareas menores	`chore(mobile): actualiza expo a 58.0.19`

Tipo	Cuándo usarlo	Ejemplo
`revert`	Deshace un commit	`revert: deshace "feat(backend): agrega serie por hora"`

Alcances habituales: `backend`, `web`, `mobile`, `iot`, `mosquitto`, `db` o el nombre de una app (`alertas`). Un cambio que rompe compatibilidad se marca con `!` (por ejemplo `feat(backend)!:`) y se explica en el cuerpo. Se evitan mensajes como `cambios` o `varios arreglos`.

8.5 Antes de abrir un pull request

Rama	Comprobaciones que deben pasar	Comandos
`backend`	Pruebas, estilo y migraciones al día	`cd BackEnd && .venv/bin/python manage.py test --noinput`, `.venv/bin/ruff check .`, `manage.py makemigrations --check --dry-run`
`frontend`	Lint y compilación	`cd FrontEnd && npm run lint && npm run build`
`mobile`	Lint y paquete de Android	`cd Mobile && npm run lint && npm run export:android`
`iot`	Pruebas de lógica del firmware y, si cambia Mosquitto, ACL y certificados	`IoT/.venv/bin/python -m unittest discover -s IoT/tests -t IoT`, `bash Mosquitto/preparar.sh`

8.6 Acceso a GitHub y seguridad

- Para subir cambios se necesita una cuenta con permiso de escritura y una credencial: una clave SSH o un token personal. Un token **no se escribe en la URL del remoto, en archivos ni en el código**; se pide al subir o se guarda en el gestor de credenciales del sistema.
- Los tokens de un solo uso se revocan al terminar la tarea (en GitHub: Settings → Developer settings → Personal access tokens).
- Nunca se sube un `.env`, un `secrets.py`, un certificado ni una clave privada. Si ocurre, se cambia el secreto de inmediato y se avisa.

8.7 Estado actual del repositorio

Elemento	Estado
Remoto	`origin` = `https://github.com/BrandonGlz/Nucleo.git`
Ramas publicadas	`main`, `dev`, `backend`, `frontend`, `mobile` e `iot`
Historial	Empieza con `chore: versión inicial del proyecto Núcleo`; los commits siguen la guía de la sección 8.4 y van a nombre de quien los hizo
Tags	Ninguno todavía; el primero se crea al cerrar la primera etapa
Protección de ramas e integración continua	No configuradas; se recomienda proteger `main` y `dev` (exigir pull request y prohibir el push forzado) y ejecutar las pruebas del área en cada pull request con GitHub Actions
Guía en el repositorio	README principal, sección «Control de versiones»

9. Pendientes para integrar al SRS

Lista de temas que el SRS todavía debe definir o precisar para poder **terminar el backend, el IoT y Mosquitto**, más lo necesario para las pruebas locales con la base de datos. Cada fila indica lo que existe hoy y la decisión que falta. Prioridad **Alta** = bloquea código o despliegue; **Media** = afina el comportamiento; **Baja** = mejora.

9.1 Backend

#	Tema	Estado actual	Qué debe definir el SRS	Prioridad
B1	Proveedor y reglas de la IA	La redacción de reportes está preparada pero sin proveedor: la API responde 503.	Proveedor y modelo; costo y límites de uso; qué datos se pueden enviar (privacidad de la organización); idioma y tono; tiempo máximo de espera y qué hacer si falla (reintentar, enviar sin texto)	Alta
B2	Contenido y diseño del reporte	El PDF trae cifras, una curva por variable, picos y conteo por nivel; el texto de IA no existe aún.	Secciones obligatorias; tipos (diario, semanal, mensual, por área o dispositivo); tipos de gráfica; logotipo y estilo; idioma; límite de periodo; cómo se nombra y cuánto se conserva el archivo	Alta
B3	Agente de IA de consulta	No existe. El SRS lo menciona para consultar datos y pedir reportes.	Alcance (solo lectura), qué consultas admite, límites por rol y organización, historial de conversación, endpoint y formato de respuesta, proveedor intercambiable, controles contra fugas entre organizaciones	Alta
B4	Proveedor y plantillas de correo	Correo por SMTP configurable; en desarrollo se imprime en consola.	Proveedor; remitente y dominio (SPF/DKIM); plantillas de alerta y de reporte; frecuencia máxima y agrupación; destinatarios de cada tipo de aviso	Alta
B5	Notificaciones push	Implementadas con Expo y apagadas; se envían a los teléfonos de los responsables.	Cuándo se envía (por nivel), texto, sonido y prioridad, agrupación, horarios de silencio, quién la recibe; credenciales FCM y APNs; qué ocurre al abrir la notificación	Media
B6	Matriz de permisos por rol	Implementada en el código (secciones 5.2 y 5.3) y probada, pero no escrita como tabla del SRS.	Tabla formal rol × recurso × acción; casos límite: analista o supervisor sin área, operador sin dispositivo, quién edita umbrales, quién crea áreas y usuarios	Alta
B7	Ciclo de vida de dispositivos y sensores	Alta y baja por API (ADMIN); estados de dispositivo y de sensor; sin flujo de reemplazo.	Quién asigna el código MQTT y las credenciales; qué pasa con el histórico al dar de baja o reemplazar un sensor; quién marca `CALIB` y `FALLA`; qué hace la ingesta con dispositivos en mantenimiento o de baja	Alta
B8	Reglas de alertas	Una alerta pendiente por umbral y nivel; escala de advertencia a peligro; se resuelve manualmente.	Anti-rebote o histéresis; recordatorios y escalamiento si no se atiende; tiempo máximo de atención; si «Sin conexión» y «Batería baja» generan alerta; qué se hace con alertas viejas	Alta
B9	Detección de «Sin conexión»	`marcar_sin_conexion` marca tras 3 intervalos sin datos; hay que ejecutarlo periódicamente.	Factor oficial; si avisa a alguien; quién lo ejecuta y cada cuánto; relación con el aviso inmediato del broker	Media
B10	Cuentas y sesión	Token de 7 días sin uso, límite de 10 inicios por minuto; sin recuperación de contraseña ni bloqueo.	Política de contraseñas; recuperación por correo; bloqueo por intentos fallidos; segundo factor; cierre de sesión global; límite de teléfonos por usuario	Media
B11	Auditoría	No existe bitácora de cambios.	Qué acciones se registran (umbrales, usuarios, dispositivos, atención de alertas), quién puede consultarla y cuánto se conserva	Media
B12	Zona horaria e idioma por organización	Zona única `America/Tijuana`; textos en español.	Si cada organización tiene su zona y formato; idiomas a soportar	Baja
B13	Retención y archivos	Las lecturas, alertas y reportes crecen sin límite; los PDF se guardan en disco local.	Tiempo de conservación por tipo de dato; dónde se guardan los PDF (disco, almacenamiento de objetos); borrado al dar de baja una organización	Alta
B14	Procesos de fondo en producción	`escuchar_mqtt`, `marcar_sin_conexion` y `generar_reportes_programados` se lanzan a mano.	Cómo se ejecutan y supervisan (servicio del sistema, contenedor, cron), reinicio automático y alerta si el consumidor se cae	Alta

#	Tema	Estado actual	Qué debe definir el SRS	Prioridad
B15	Requisitos no funcionales medibles	El SRS pide 2 minutos máximo para la alerta; no hay otras metas numéricas.	Dispositivos y usuarios simultáneos; lecturas por segundo; latencia de lectura a alerta y a pantalla; disponibilidad; tiempos de respuesta de la API y de generación de reportes; límites de peticiones	Alta
B16	Observabilidad	Registros de aplicación en consola.	Registros centralizados, métricas y avisos operativos (broker, Redis, base de datos, consumidor)	Media

9.2 IoT (firmware y hardware)

#	Tema	Estado actual	Qué debe definir el SRS	Prioridad
I1	Hardware definitivo	Pines y circuito provisionales; ESP32 DevKit genérico.	Modelo exacto del ESP32; pinout final; alimentación y carga de la batería; divisor de tensión y protecciones; caja y ubicación de sensores	Alta
I2	Calibración y significado del aire	El CCS811 entrega eCO2 estimado; el baseline se guarda cada hora; sin validar con el sensor real.	Cómo se calibra y cada cuánto; si se mide también TVOC (no existe como variable); umbrales por defecto de cada variable y por contexto de uso	Alta
I3	Modo sin conexión	Cola de 20 mensajes en memoria (se pierde al reiniciar); alertas locales con LED y buzzer.	Capacidad y persistencia (¿memoria flash?); política de reenvío y orden; tiempo máximo; comportamiento sin hora NTP	Media
I4	Actualización del firmware	No existe; se sube por USB.	Si se necesita OTA, cómo se versiona y se distribuye, y cómo se revierte	Media
I5	Aprovisionamiento en campo	Wi-Fi y credenciales en `secrets.py` cargado por USB.	Cómo se configura un dispositivo nuevo sin conectar una computadora (portal cautivo, Bluetooth, tarjeta) y cómo se rotan las credenciales	Media
I6	Indicadores locales	LED RGB verde/amarillo/rojo, buzzer a pulsos en peligro y pantalla OLED.	Significado exacto de cada indicador; si se puede silenciar el buzzer; contenido y rotación de pantallas; idioma	Baja
I7	Intervalos y vigilancia	Intervalo por defecto 30 s (5 a 120); vigilante (`WATCHDOG_MS`) desactivado.	Intervalo por defecto y límites oficiales; si se activa el reinicio por bloqueo; modo de bajo consumo y su efecto en la medición	Media
I8	Batería	Medición desactivada hasta montar el divisor; «Batería baja» al 20 %.	Umbral oficial, curva voltaje-porcentaje, aviso a quién y duración esperada	Media

9.3 Mosquitto (broker MQTT)

#	Tema	Estado actual	Qué debe definir el SRS	Prioridad
M1	Despliegue y certificados	Broker en Docker con CA y certificados de desarrollo.	Dónde corre (servidor de producción), dominio o IP, certificado emitido por una CA reconocida o propia, rotación, puertos y reglas de firewall	Alta
M2	Credenciales por dispositivo	Un usuario por dispositivo con `dispositivo.sh`, a mano; la ACL usa `%u`.	Flujo de alta, rotación y baja desde la plataforma; revocar al dar de baja; dónde se muestran las credenciales una sola vez; si se usa un mecanismo dinámico en vez de archivo	Alta
M3	Calidad de servicio y mensajes	QoS 1 para lecturas; configuración retenida; detección de caída por silencio.	Mensaje de última voluntad (LWT) para detectar caídas al instante; tamaño y frecuencia máximos; persistencia del broker; versión del formato del mensaje	Media
M4	Comandos hacia el dispositivo	Solo viaja la configuración (intervalo y umbrales).	Si habrá comandos (reiniciar, silenciar, actualizar) y su tópico, formato y confirmación	Baja
M5	Capacidad y alta disponibilidad	Un solo broker sin clúster.	Dispositivos concurrentes esperados y si se exige redundancia	Media

9.4 Base de datos y pruebas locales

#	Tema	Estado actual	Qué debe definir el SRS	Prioridad
D1	Datos de demostración y volumen de prueba	`sembrar_demo` crea una organización, un usuario por rol y un dispositivo; sin histórico.	Conjunto oficial de datos semilla; cuántos días de lecturas se necesitan para probar gráficas y reportes; herramienta para simular uno o varios dispositivos	Alta
D2	Entornos y nombres	Desarrollo con superusuarios de contenedor; `crear-roles.sh` listo y sin aplicar.	Entornos (desarrollo, pruebas, producción), nombres de bases y usuarios, quién ejecuta migraciones y cómo en el despliegue	Alta
D3	Reglas en la base vs. en el código	Hay `CHECK`, `UNIQUE` e índices; `procedures.sql` y `triggers.sql` vacíos.	Qué reglas (límite de 2 minutos, destinatarios, una alerta pendiente) deben vivir también como procedimientos o triggers	Media
D4	TimescaleDB: retención y rendimiento	Hypertable por día sin compresión, retención ni agregados.	Compresión, retención, agregados continuos para series y reportes, y tamaño de bloque	Media
D5	Respaldos y recuperación	Comando manual de `pg_dump` documentado.	Frecuencia, destino, cifrado, pruebas de restauración y metas de pérdida y recuperación	Alta
D6	Scripts de práctica	Copia manual del modelo.	Quién los mantiene sincronizados con las migraciones, o si se generan automáticamente	Baja

9.5 Transversales

#	Tema	Estado actual	Qué debe definir el SRS	Prioridad
G1	Despliegue en servidor (Vultr)	Nada desplegado.	Servidor y recursos; HTTPS y proxy inverso con WebSocket; Redis y bases fuera de la red pública; gestión de secretos; dominio	Alta
G2	Pantallas de web y móvil	Esqueleto de módulos sin diseño.	Bocetos o flujos por rol y módulo, textos, estados vacíos y de error, accesibilidad y reglas del modo claro y oscuro	Alta
G4	Revisión e integración continua	Repositorio publicado con ramas por área; sin protección de ramas ni pruebas automáticas en cada pull request.	Quién revisa y aprueba los pull requests; pruebas obligatorias por área (GitHub Actions); protección de `main` y `dev`; nombre y criterio de cierre de cada etapa (tags)	Media
G3	Pruebas de aceptación	Pruebas automáticas del backend y del firmware; ninguna de interfaz.	Criterios de aceptación por requisito y quién ejecuta las pruebas con hardware real	Media

9.6 Orden sugerido para cerrar la brecha

- **Primero lo que bloquea código:** proveedor de IA y diseño del reporte (B1, B2), proveedor de correo (B4), matriz de permisos (B6), ciclo de vida de dispositivos y credenciales MQTT (B7, M2) y hardware definitivo (I1).
- **Después lo que fija el comportamiento:** reglas de alertas (B8), sin conexión (B9), calibración y umbrales (I2), entornos y datos de prueba (D1, D2).
- **Antes de producción:** despliegue y certificados (G1, M1), procesos de fondo (B14), retención y respaldos (B13, D5), metas no funcionales (B15) y observabilidad (B16).
- **Cuando haya tiempo:** actualización OTA, aprovisionamiento, comandos al dispositivo, auditoría, idioma y zona por organización.

Con estas decisiones en el SRS, lo que queda es implementar: el agente de IA y el cliente del proveedor, las plantillas de correo, el flujo de alta de dispositivos, las pantallas de web y móvil, y probar el firmware en la placa.